

Agentic CI/CD Pipeline Repair & Intelligent Release Orchestration

Introduction

Tanmay Dattatray Raut – 23121007 Ganesh Vikas Sonawane – 23121019 Sakshi Ramdas Gore – 24221078

Guided by: Dr. Danny Pariera

1. Introduction

Continuous Integration and Continuous Deployment (CI/CD) have become an important part of modern software engineering by automating the integration, building, testing, and delivery of software changes. Continuous Integration enables developers to frequently integrate changes into a shared repository, where automated builds and tests provide early feedback about defects. Continuous Delivery and Continuous Deployment extend this process by preparing validated changes for release and, when the required conditions are satisfied, delivering them to the target environment. These practices improve development speed, consistency, and release frequency by reducing repetitive manual operations. However, the increasing complexity of software projects, dependency environments, and deployment workflows also increases the difficulty of recovering from pipeline failures. A failed pipeline may result from programming defects, dependency conflicts, configuration errors, failed tests, infrastructure problems, or transient execution issues. Although CI/CD systems can identify that a workflow has failed, developers often still need to inspect logs, test reports, source code, commit history, and changed files to determine the actual cause. Consequently, failure recovery remains a time-consuming activity, particularly when failures occur frequently or when the available diagnostic information is distributed across multiple tools.

To reduce this manual effort, several techniques have been explored for CI/CD failure analysis and automated software repair. Traditional monitoring and rule-based approaches detect failures by examining workflow status, error messages, and predefined conditions. Automated Program Repair (APR) techniques generate patches for software defects, while Large Language Models (LLMs) provide additional capabilities for code understanding, debugging, and patch generation. More recent approaches combine these capabilities with Retrieval-Augmented Generation (RAG), which retrieves relevant information from historical failures, source code, or other knowledge bases before generating a response. For example, LogSage presents an LLM-based framework for CI/CD failure detection and remediation that performs log preprocessing, root-cause analysis, and solution generation using RAG and tool-calling [1]. Another study investigates a practical CI-repair pipeline involving failure-type classification, retrieval-based repair, and constrained patch generation for GitHub Actions workflows [2]. These approaches demonstrate that AI can assist with both understanding failures and generating possible repairs. However, the reliability of a repair still depends on the quality of the retrieved context, the correctness of the generated patch, and the ability to verify that the proposed change actually resolves the failure.

Despite these advances, an important research and engineering gap remains in connecting failure diagnosis, automated repair, repair validation, and release decision-making into a unified workflow. Existing approaches may concentrate on log analysis, historical-fix retrieval, or patch generation, while the complete process of determining whether a generated repair is safe to release requires additional validation and governance. A patch that appears syntactically correct may still fail functional tests, introduce regressions, or modify code that is critical to the application. Furthermore, different changes may require different release strategies depending on their scope and risk. A minor configuration change and a change affecting critical application logic should not necessarily be treated in the same way. Therefore, a system that automatically generates a patch without validating its behavior or assessing its release risk cannot be considered a complete self-healing CI/CD solution. The research gap addressed by this project is the need for an integrated, context-aware repair workflow that not only identifies the likely cause of a pipeline failure, but also generates a targeted repair, verifies the repair through re-execution, and supports an appropriate release decision while retaining human oversight for high-risk changes.

To address this gap, this project proposes an Agentic CI/CD Pipeline Repair and Intelligent Release Orchestration system that integrates Large Language Models, Retrieval-Augmented Generation, GitHub Actions, automated patch generation, and risk-based governance into a unified workflow. When a pipeline failure occurs, the system collects and normalizes relevant logs, identifies failing tests, and retrieves important source-code and commit information to provide the AI agent with project-specific context. The agent then performs failure diagnosis and generates a small, targeted patch intended to address the identified problem without unnecessarily affecting unrelated parts of the application. The proposed system subsequently applies the repair and executes a validation loop to determine whether the pipeline succeeds. If the repair fails, the system can retry the process within a bounded number of attempts or escalate the issue for human intervention. A pull request containing the proposed fix, explanation, and relevant logs is generated to support review and traceability. Finally, a governance component assesses the importance of the affected code and the extent of the change to determine whether the repair can proceed automatically or requires approval. Thus, the contribution of this project is the integration of context-aware failure diagnosis, targeted automated repair, repair-validation feedback, and risk-based release orchestration within a single CI/CD workflow. The proposed approach aims to reduce manual debugging effort and mean time to recovery while improving release reliability and maintaining human control over risky changes.

References

- [1] W. Xu, J. Luo, T. Huang, K. Sui, J. Geng, Q. Ma, I. Akasaka, X. Shi, J. Tang, and P. Cai, “A Two-Staged LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation,” in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, 2025, pp. 3742–3753. doi: 10.1109/ASE63991.2025.00310.
- [2] H. Zhang, “LLM-Driven CI Failure Diagnosis and Automated Repair: From GitHub Actions Logs to Patch Recommendation,” *Journal of Technology Informatics and Engineering*, vol. 4, no. 1, pp. 190–214, 2025. doi: 10.51903/jtie.v4i1.484.